Guía de Trabajo

Desarrollo Web Full Stack Frontend — TypeScript en React

Semana 9 | Proyecto integrador: Plataforma Web de Experiencias Interactivas

Producto articulador: una tarjeta de experiencia tipada con TypeScript y animada con Framer Motion.

1. Datos generales

Elemento	Descripción
Semana	**9**
Tema	**TypeScript en React: interfaces, props, estado y custom hooks**
Modalidad	**Trabajo práctico en equipo**
Duración sugerida	**2 a 3 horas**
Requisitos previos	**Aplicación React de la semana 8; componentes, props, useState y estructura básica del proyecto**
Entregable	**Componentes principales migrados a TypeScript y una Card de experiencia tipada**

2. Situación problemática

La Plataforma Web de Experiencias Interactivas presenta tarjetas para mostrar lugares, eventos o productos. Al trabajar solo con JavaScript, un componente podría recibir una valoración como texto, omitir un título requerido o recibir una categoría no contemplada. El equipo necesita definir contratos explícitos para que los componentes sean más predecibles y fáciles de mantener.

3. Resultado de aprendizaje

Al finalizar, el estudiante define tipos e interfaces para los datos y las props de React, aplica tipado a estado y funciones, y migra al menos un componente reutilizable a TypeScript justificando cómo mejora su mantenibilidad.

4. Conceptos clave

Concepto	Uso en el proyecto	Ejemplo
Type	**Describe un tipo, unión o alias**	**type Category = "evento" | "lugar" | "producto"**
Interface	**Define la forma de un objeto**	**interface Experience { id: number; title: string }**
Props	**Contrato entre componente padre e hijo**	**interface ExperienceCardProps { item: Experience }**
Estado	**Datos que cambian durante la interacción**	**const [favorite, setFavorite] = useState<boolean>(false)**
Tipo opcional	**Propiedad que puede no existir**	**imageUrl?: string**
Custom hook	**Lógica reutilizable con tipos claros**	**function useFavorites() { … }**

5. Preparación del entorno

1. Abre el repositorio de la Plataforma Web de Experiencias Interactivas desarrollado hasta la semana 8.
2. Verifica que la aplicación se ejecute correctamente antes de hacer cambios.
3. Si el proyecto aún está en JavaScript, confirma con el docente el procedimiento de conversión a TypeScript. En un proyecto Vite React nuevo, selecciona la plantilla react-ts.
4. Instala dependencias del proyecto según el package manager utilizado y registra los cambios en una rama de trabajo.

Importante: no cambies toda la aplicación de una sola vez. Migra primero un componente pequeño, comprueba que compile y continúa progresivamente.

6. Actividades

Actividad 1. Definir el modelo de datos

Crea src/types/experience.ts y define una interfaz que represente una experiencia de la plataforma.

Código de referencia:

```
export type ExperienceCategory = "evento" | "lugar" | "producto";

export interface Experience {
  id: number;
  title: string;
  description: string;
  category: ExperienceCategory;
  rating: number;
  imageUrl?: string;
}
```

Personaliza los campos de acuerdo con el modelo real del proyecto. Evita duplicar interfaces equivalentes en distintos archivos.

Actividad 2. Tipar las props de una tarjeta

Crea o migra ExperienceCard.tsx. El componente debe recibir una experiencia y mostrar título, descripción, categoría y valoración. Si imageUrl no está disponible, debe presentar una alternativa visual coherente.

```
import type { Experience } from "../types/experience";

interface ExperienceCardProps {
  item: Experience;
  onSelect?: (id: number) => void;
}

export function ExperienceCard({ item, onSelect }: ExperienceCardProps) {
  return (
    <article className="experience-card">
      {item.imageUrl && <img src={item.imageUrl} alt="" />}
      <p>{item.category}</p>
      <h2>{item.title}</h2>
      <p>{item.description}</p>
      <p aria-label={`Valoración: ${item.rating} de 5`}>
        {item.rating}/5
```

```
      </p>
      <button onClick={() => onSelect?.(item.id)}>
        Ver experiencia
      </button>
    </article>
  );
}
```

Nota: el atributo alt vacío solo es adecuado si la imagen es decorativa y la información ya está disponible en el texto. Si la imagen aporta información, usa un texto alternativo descriptivo.

Actividad 3. Tipar estado y eventos

Incorpora un estado de favorito. El botón debe cambiar su etiqueta y estado al pulsarse.

```
const [isFavorite, setIsFavorite] = useState<boolean>(false);

<button
  type="button"
  aria-pressed={isFavorite}
  onClick={() => setIsFavorite(current => !current)}
>
  {isFavorite ? "Quitar de favoritos" : "Agregar a favoritos"}
</button>
```

Actividad 4. Tipar una colección

Declara los datos de prueba como Experience[] y renderiza las tarjetas mediante map. Asegúrate de usar una key estable y única.

```
const experiences: Experience[] = [
  {
    id: 1,
    title: "Ruta de arte urbano",
    description: "Recorrido por murales y espacios creativos.",
    category: "lugar",
    rating: 4.6
  }
];

// En el JSX:
// {experiences.map(item => <ExperienceCard key={item.id} item={item} />)}
```

Actividad 5. Migrar un componente existente

- Selecciona un componente principal de la aplicación que todavía esté en JavaScript.
- Renombra el archivo a .tsx cuando contenga JSX y define los tipos de sus props.
- Tipa los estados cuando el tipo no pueda inferirse con claridad.
- Corrige errores de compilación sin recurrir a any como solución automática.
- Comprueba que el diseño, las interacciones y la navegación continúen funcionando.

7. Reto de aplicación

Implementa una Card de experiencia tipada que pueda reutilizarse para mostrar por lo menos tres elementos distintos del catálogo. Debe aceptar una función opcional para seleccionar la experiencia y manejar correctamente los campos opcionales.

8. Pruebas y criterios de aceptación

Prueba	Resultado esperado	Cumple
Compilación	**El proyecto compila sin errores de TypeScript**	☑
Datos	**Los objetos respetan la interfaz Experience**	☑
Props	**La tarjeta recibe props tipadas y reutilizables**	☑
Estado	**El control de favorito cambia correctamente**	☑
Campos opcionales	**La tarjeta funciona cuando no hay imageUrl**	☑
Regresión	**La aplicación mantiene su comportamiento anterior**	☑
Código	**No se usa any indiscriminadamente ni se duplican tipos**	☑

9. Entregables

- Repositorio actualizado con los cambios de TypeScript.
- Archivos de tipos e interfaces utilizados.
- Componente ExperienceCard tipado e integrado en la aplicación.
- Capturas o breve demostración del componente funcionando.
- Comentario breve (150–250 palabras): dos errores que TypeScript ayuda a detectar y una limitación que todavía requiere pruebas.
- Video de exposición de 8 a 10 minutos.

10. Preguntas de reflexión

1. ¿Qué error concreto podría detectarse antes de ejecutar gracias a la interfaz Experience?
2. ¿Cuándo conviene usar interface y cuándo un type alias?
3. ¿Por qué es riesgoso usar any para resolver rápidamente un error?
4. ¿Qué aspectos del componente no pueden validarse únicamente con TypeScript?
5. ¿Cómo se coordinará este componente con la animación que se realizará en el laboratorio inmersivo?

11. Cierre y articulación con el laboratorio

La entrega de Desarrollo Web es una tarjeta con estructura y datos confiables. En el Laboratorio de Experiencias Web Inmersivas, el equipo añadirá transiciones y microinteracciones con Framer Motion sin alterar el contrato de datos ni perjudicar accesibilidad o rendimiento.

Referencias técnicas

- **TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html**
- **React — Using TypeScript: https://react.dev/learn/typescript**
- **Vite — Getting Started: https://vite.dev/guide/**

Anexo de la entrega

Semana 9 · TypeScript en React · Wayra Academy

Comentario breve

TypeScript ayuda a detectar errores antes de ejecutar la aplicación. El primero apareció durante esta migración: al declarar `CURSOS` como `Experience[]`, el compilador señaló los 16 cursos porque aún no tenían el campo obligatorio `valoracion`. En JavaScript, ese fallo se habría visto recién en el navegador, al intentar mostrar la nota de una tarjeta. El segundo error es el uso de valores no contemplados: como `categoria` es una unión de cuatro textos, escribir `"disenio"` no compila, mientras que antes habría producido una etiqueta vacía sin ningún aviso. También evita acceder a valores que pueden no existir, como el resultado de `CURSOS.find`.

Sin embargo, TypeScript solo revisa el código, no lo que sucede al ejecutarlo. El tipo `number` acepta una valoración de 9 aunque la escala llegue a 5, y los datos leídos de `localStorage` pueden tener cualquier forma; por eso `useFavoritos` los valida al leerlos. Tampoco garantiza que el botón sea accesible, que la imagen se vea bien o que el favorito se conserve al filtrar. De hecho, un recorte incorrecto de la imagen pasó la compilación y solo se descubrió al revisar las capturas. Esas conductas todavía requieren pruebas funcionales y visuales.

Respuestas a las preguntas de reflexión

1. **¿Qué error concreto se detecta gracias a `Experience`?** Un curso sin `valoracion`, una valoración escrita como texto (`"4.5"`) o una categoría que no existe. Los tres fallan al compilar.

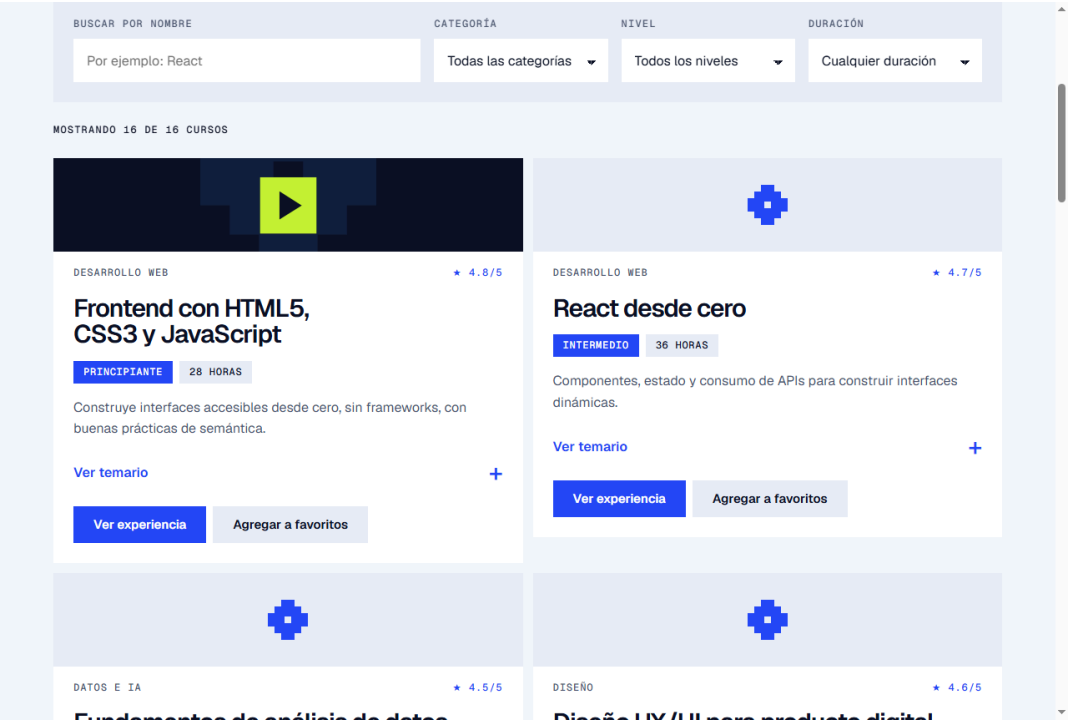
2. **¿Cuándo usar `interface` y cuándo `type`?** `interface` para la forma de un objeto, como `Experience` o las props. `type` para uniones y combinaciones, como `ExperienceCategory` o `BotonProps`.

3. **¿Por qué es riesgoso `any`?** Apaga la comprobación y el error reaparece al ejecutar, cuando ya lo ve el usuario. Además se propaga a todo lo que usa ese valor. Si el tipo se desconoce, es mejor `unknown` y validarlo, como hace `useFavoritos`.

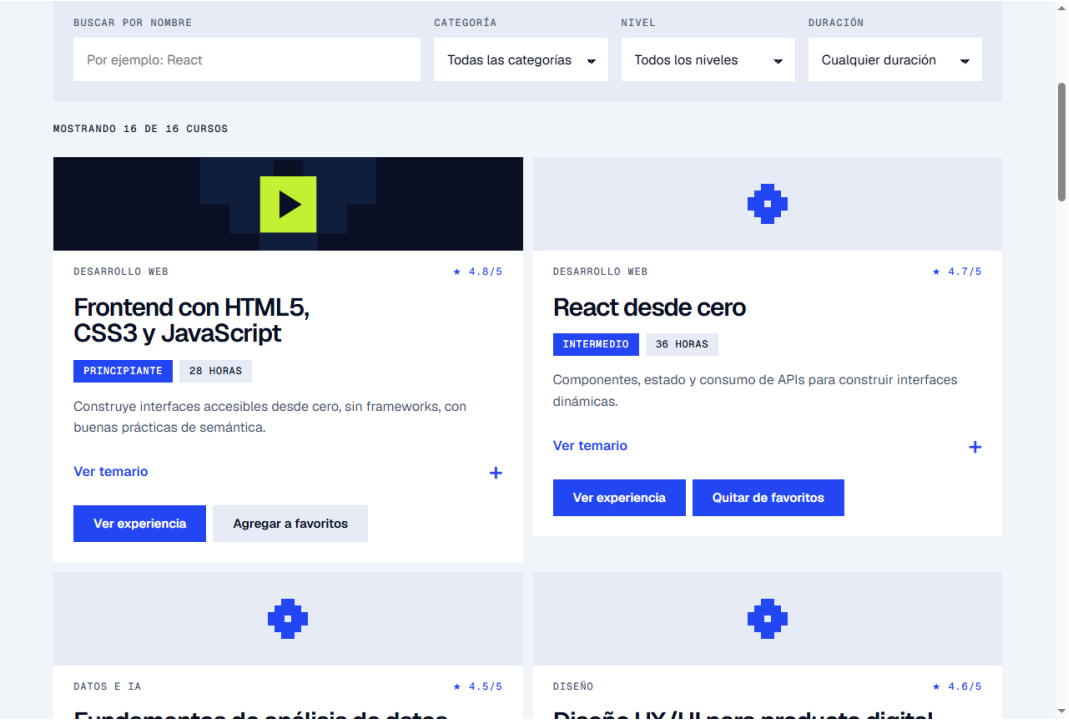
4. **¿Qué no se puede validar solo con TypeScript?** Rangos de valores, datos externos, accesibilidad real, diseño y comportamiento. Las conversiones con `as` en los filtros tampoco se comprueban al ejecutar: funcionan porque las opciones del `<select>` están escritas a mano.

5. **¿Cómo se coordina con el laboratorio inmersivo?** Framer Motion se añadirá por fuera del contrato: `Experience` y las props actuales no cambian. Habrá que reemplazar la animación CSS de entrada (`--i`) en lugar de sumarle otra, y respetar «reducir movimiento».

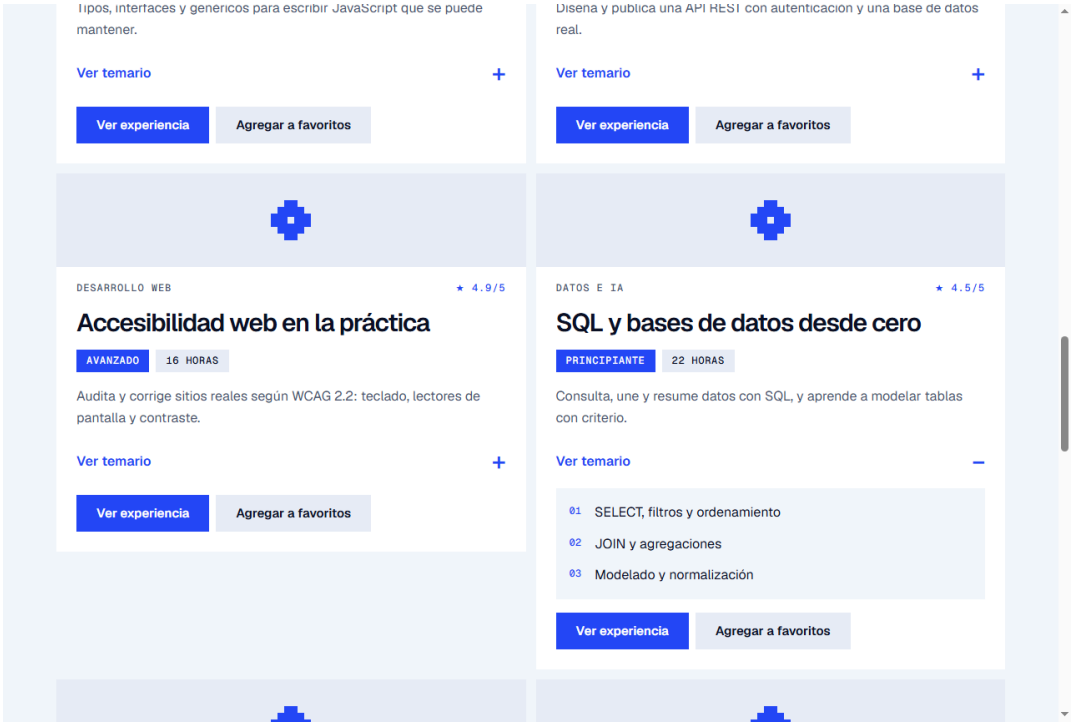
Capturas del componente funcionando



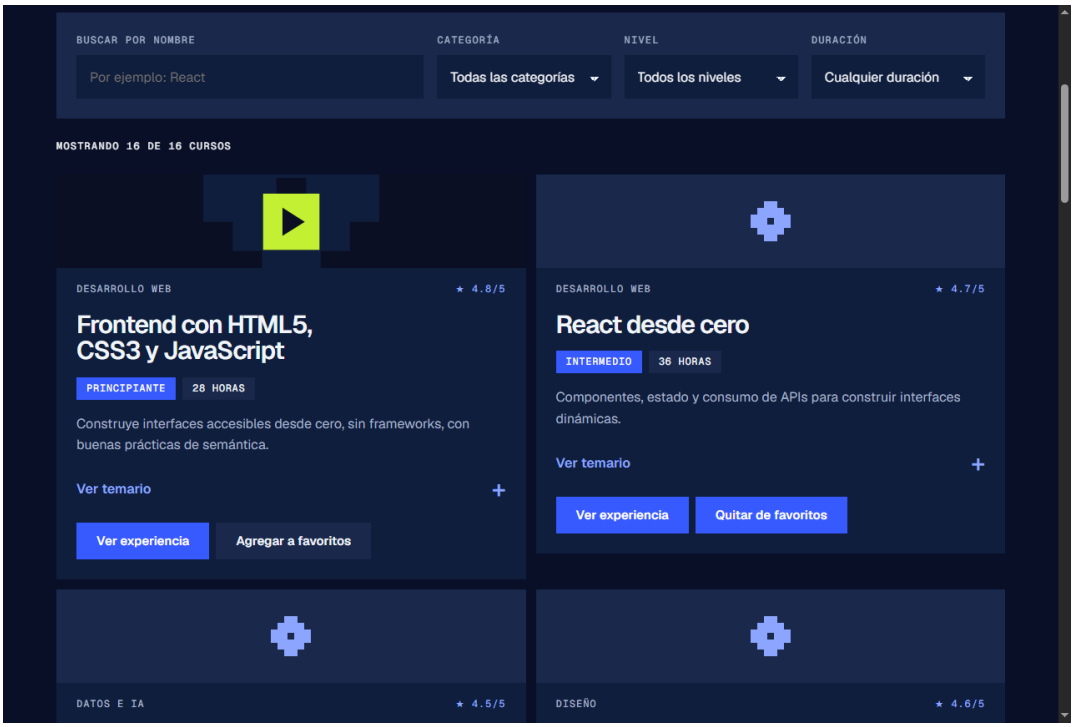
Tarjeta con imagen y tarjeta con la alternativa visual (sin imagen).



Favorito activo: la etiqueta cambia a «Quitar de favoritos».



«Ver experiencia» abre el temario y deja el curso en la URL.



La tarjeta en tema oscuro.